

MCT-453L: Machine Vision

Design and Evaluation of a Deep Learning-Based Vehicle Detection and Classification System Using EfficientNet-B0 and YOLOv8



Session: 2022

SUBMITTED TO

Dr. Zulqarnain Naqvi

SUBMITTED BY

Muhammad Bilal Afzal Awan	2022-MC-41
Muhammad Haider Ali	2022-MC-45
Muhammad	2022-MC-48
Muhammad Abdullah	2022-MC-50
Azmeer Ali	2022-MC-53
Faiq Ahmad Zia	2022-MC-59
Daies Ali Shah	2022-MC-67

**Department of Mechatronics & Control Engineering, University of
Engineering & Technology, Lahore, Pakistan**

List of Figures

Figure 1. Data Augmentation	38
Figure 2. Class Distribution	38
Figure 3. Confusion Matrix – EfficientNet-B0.....	39
Figure 4. EfficientNet-B0 Train/Val Loss vs Train/Val Accuracy	39
Figure 5. Architecture Comparison	40
Figure 6. Training Optimization Comparison	41
Figure 7. Deployment Inference Analysis	41
Figure 8. Confusion Matrix - YOLOv8n	42
Figure 9. Confusion Matrix Normalized - YOLOv8n	43
Figure 10. YOLOv8n Metrics.....	43
Figure 11. YOLOv8n F1 Curve.....	44
Figure 12. YOLOv8n Precision Curve	45
Figure 13. YOLOv8n Precision-Recall Curve	46
Figure 14. YOLOv8n Recall Curve	47
Figure 15. Gradio Inference - Phase 5	47
Figure 16. YOLOv8n Predictions on test batch 1	48
Figure 17. YOLOv8n Predictions on test batch 2	49

List of Tables

Table 1. Performance Summary Comparison	4
Table 2. Hyperparameters.....	7
Table 3. EfficientNet-B0 Metrics.....	9
Table 4. YOLOv8n Metrics	10
Table 5. Comparative Analysis - EfficientNet-B0 vs YOLOv8n	10
Table 6. Computational Performance - EfficientNet-B0 vs YOLOv8n	11
Table 7. Optimizer Results.....	12
Table 8. Model Limitations - EfficientNet-B0 vs YOLOv8n.....	15

Chapter 1

Executive Summary

Problem Overview

This project designs and evaluates a deep learning-based computer vision system for automated vehicle detection and classification in real-world traffic imagery. Six vehicle categories are targeted bus, car, microbus, motorbike, pickup-van, and truck using a dataset of 9,211 annotated images sourced from Roboflow [1]. Two architectures are implemented and compared: EfficientNet-B0 for classification and YOLOv8n for object detection, across a six-phase pipeline covering dataset engineering, training, optimization, deployment, and critical analysis.

Key Findings

EfficientNet-B0 achieved 81.2% validation accuracy through transfer learning, while YOLOv8n achieved 73.0% mAP@50 on the test set. Training optimization experiments showed SGD with StepLR and dropout 0.3 yielded the highest accuracy (81.86%), while AdamW with dropout 0.5 generalized best with the lowest overfitting gap. Both models achieved near-identical inference speeds of ~37ms per image on a Tesla T4 GPU. Larger vehicles (bus, truck) were consistently easier to detect; motorbikes and microbuses posed the greatest challenge due to small size and visual similarity to adjacent classes.

Performance Summary

Table 1. Performance Summary Comparison

Metric	EfficientNet-B0	YOLOv8n
Task	Classification	Detection
Accuracy / mAP@50	81.2%	73.0%
Inference Time	36.8ms	36.6ms
Parameters	5.3M	3.0M
Weakest Class	Microbus (F1: 0.407)	Motorbike (AP@50: 0.633)

Chapter 2

Introduction

Problem Definition

Manual monitoring of traffic cameras is neither scalable nor cost-effective. This project addresses that gap by implementing a deep learning pipeline that automatically detects and classifies six vehicle types from traffic scene images formulated both as a classification task and an object detection task.

Motivation

Automated vehicle classification underpins intelligent transportation applications including toll collection, traffic density estimation, and urban planning. This project is further motivated by the engineering challenge of adapting pre-trained architectures to a domain-specific problem under real-world constraints: class imbalance, occlusion, viewpoint variation, and limited compute resources. Furthermore, traffic accidents, justice, using ethical code of conduct and engineering discipline. We were most motivated by our end goal which is to prevent harms to innocent and weak using technology. Many accident cases, unjustified road rages which results in deaths of innocent people over minor disputes and preventing the harassment of women and children is what our underlying ultimate goal is.

Related Work

EfficientNet (Tan & Le, 2019) [2] established that compound scaling of depth, width, and resolution yields strong accuracy with fewer parameters, making it well-suited for transfer learning. On the detection side, the YOLO family (Redmon et al., 2016) [3] evolved from two-stage pipelines toward fast single-stage detection, with YOLOv8n by Ultralytics [4] representing the current state of the art for resource-constrained deployment. Prior work confirms that fine-tuned CNNs perform well on curated traffic datasets but degrade under occlusion, lighting variation, and class imbalance which are the conditions directly addressed in this project through augmentation and regularization.

Chapter 3

Methodology

Dataset Preparation

The dataset was sourced from Roboflow (Vehicle Detection Dataset v3, CC BY 4.0), containing 9,211 annotated traffic images across six vehicle classes: bus, car, microbus, motorbike, pickup-van, and truck. Images were downloaded in YOLOv8 format. The dataset was split 70% training (6,449), 15% validation (1,381), and 15% test (1,381) using a fixed random seed of 42.

Class imbalance is significant: car accounts for 4,295 test instances while truck has only 229. To improve generalization, the following augmentations were applied:

- Random Horizontal Flip ($p=0.5$) simulates bidirectional traffic
- Random Rotation ($\pm 10^\circ$) accounts for camera tilt
- Color Jitter (brightness, contrast, saturation) simulates lighting variation
- Random Grayscale ($p=0.1$) simulates low-quality or IR cameras
- Normalize with ImageNet mean/std required for pretrained model

For EfficientNet, a custom dataset class cropped individual vehicle instances from full scene images using bounding box coordinates, yielding 40,466 training crops, 8,611 validation crops, and 9,245 test crops. YOLOv8 was trained on full scene images directly.

Model Architecture

EfficientNet-B0 (Baseline)

EfficientNet-B0 was loaded with ImageNet pretrained weights. All backbone layers were frozen; only the classifier head was replaced with a Dropout layer ($p=0.3$) followed by a Linear layer mapping 1,280 features to 6 output classes. Total trainable parameters: 7,686. Input size: 224×224 .

YOLOv8n (Advanced Architecture)

YOLOv8n (nano) is a single-stage object detector comprising 73 layers and 3.0M parameters. It uses a CSPDarknet backbone, PANet neck for multi-scale feature fusion, and a decoupled detection head. Pretrained on COCO, it was fine-tuned on the vehicle dataset at 640×640 input resolution. The model simultaneously predicts bounding boxes, scores, and class labels in a single forward pass.

Training Pipeline

EfficientNet-B0

Trained for 12 epochs with batch size 32 on a Tesla T4 GPU.

Optimizer: AdamW (lr=1e-4, weight_decay=1e-4).

Scheduler: CosineAnnealingLR (T_max=10).

Loss: CrossEntropyLoss. Best model checkpoint saved based on validation accuracy.

Training time: 42.4 minutes.

YOLOv8n

Trained for 12 epochs with batch size 32.

Optimizer: AdamW (auto-selected, lr=0.001, momentum=0.9).

Additional augmentations applied by Ultralytics: Blur, MedianBlur, CLAHE, ToGray.

Early stopping patience: 5.

Training time: 21.6 minutes.

Hyperparameters:

Table 2. Hyperparameters

Parameter	EfficientNet-B0	YOLOv8n
Epochs	12	12
Batch Size	32	32
Learning Rate	1e-4	0.001 (auto)
Optimizer	AdamW	AdamW
Scheduler	CosineAnnealingLR	Default (linear warmup)
Dropout	0.3	0.0
Input Size	224×224	640×640
Weight Decay	1e-4	5e-4
Loss Function	CrossEntropyLoss	Box + Cls + DFL

Evaluation Methodology

EfficientNet-B0 was evaluated on 9,245 test crops using accuracy, precision, recall, and F1-score per class, plus overall accuracy and a confusion matrix.

YOLOv8n was evaluated on 1,381 full-scene test images using mAP@50 and mAP@50-95, with per-class AP@50.

Inference speed was benchmarked over 50 test images with GPU warmup runs, measuring average latency in milliseconds.

Chapter 4

Results and Analysis

Accuracy Metrics

EfficientNet-B0 — Classification

The model achieved 81.2% best validation accuracy and 78.0% test accuracy over 9,245 crops. Training loss decreased steadily from 1.11 to 0.71 across 12 epochs, with validation loss converging at 0.61.

Table 3. EfficientNet-B0 Metrics

Class	Precision	Recall	F1-Score	Support
bus	0.839	0.764	0.800	998
car	0.785	0.906	0.841	4295
microbus	0.767	0.277	0.407	606
motorbike	0.837	0.907	0.871	2391
pickup-van	0.659	0.368	0.472	726
truck	0.722	0.533	0.613	229
Overall (weighted)	0.792	0.798	0.781	9245

Motorbike achieved the highest F1 (0.871), benefiting from its distinct shape and high sample count. Microbus (F1: 0.407) and pickup-van (F1: 0.472) performed worst due to visual similarity with adjacent classes and low representation in the dataset.

YOLOv8n — Object Detection

YOLOv8n achieved mAP@50 of 0.727 and mAP@50-95 of 0.416 on the test set. Precision: 0.69, Recall: 0.68.

Table 4. YOLOv8n Metrics

Class	AP@50	Precision	Recall
bus	0.865	0.740	0.856
car	0.725	0.620	0.725
microbus	0.698	0.641	0.679
motorbike	0.633	0.659	0.558
pickup-van	0.640	0.661	0.562
truck	0.760	0.725	0.668

Bus achieved the highest AP@50 (0.865) due to its large, distinct bounding box. Motorbike was the weakest class (0.633) due to small size and frequent occlusion in dense traffic.

Comparative Analysis

EfficientNet outperforms YOLOv8 on the classification metric (81.2% vs 73.0% mAP@50), but the comparison is not strictly equivalent — EfficientNet classifies pre-cropped patches while YOLOv8 simultaneously detects and classifies vehicles in full scenes. YOLOv8 is the more practically capable model for real-world traffic monitoring.

Table 5. Comparative Analysis - EfficientNet-B0 vs YOLOv8n

Property	EfficientNet-B0	YOLOv8n
Task	Classification	Object Detection
Accuracy / mAP@50	81.2%	73.0%
Inference Time	36.8ms	36.6ms
Parameters	4.0M	3.0M
GPU Memory	~84MB	~170MB
Input Size	224×224	640×640
Bounding Boxes	No	Yes
Multi-vehicle / image	No	Yes
Best For	Single crop	Full scene

Error Analysis

The most common failure patterns across both models are:

- Microbus misclassified as car or bus: visually intermediate vehicle; low recall (0.277) in EfficientNet.
- Pickup-van confused with truck or car depending on viewing angle and cropping.
- Motorbike missed by YOLOv8 in dense clusters: small bounding boxes, frequent overlap.
- Truck underperforms in EfficientNet (F1: 0.613) due to low training sample count (229 instances).

Class imbalance is the primary structural driver of these errors. The car class has 18.8x more instances than truck, which skews model priors and degrades minority class recall.

Computational Performance

Latencies below reflect CPU-mode inference during the Phase 5 throughput benchmark (100 images). The earlier GPU benchmark (50 images) showed ~36ms for both models, confirming GPU acceleration provides roughly 4-8x speedup. EfficientNet achieves higher throughput (12.9 FPS vs 3.3 FPS) due to its simpler classification-only output and smaller input resolution.

Table 6. Computational Performance - EfficientNet-B0 vs YOLOv8n

Metric	EfficientNet-B0	YOLOv8n
Avg Latency (ms)	77.3	298.8
Min Latency (ms)	58.0	237.2
Max Latency (ms)	177.0	418.5
P95 Latency (ms)	106.8	390.7
Std Dev (ms)	18.1	56.9
Throughput (FPS)	12.9	3.3
CPU Usage	38.6%	38.6%
RAM Used	2.1 GB	2.1 GB

Phase 4

Training Optimization Results

Three optimizer configurations were tested on EfficientNet-B0 to evaluate the effect of optimizer choice, learning rate scheduling, and dropout on generalization performance.

Table 7. Optimizer Results

Config	Best Val Acc	Overfit Gap	Time (min)
Baseline (AdamW + Cosine + drop=0.3)	80.94%	N/A	~45
SGD + StepLR + drop=0.3	81.92%	-7.93%	3.2
Adam + CosineAnnealing + drop=0.3	80.72%	-7.26%	3.2
AdamW + CosineAnnealing + drop=0.5	80.72%	-9.74%	3.2

SGD with StepLR achieved the highest validation accuracy (81.92%), marginally outperforming the AdamW baseline. The negative overfitting gaps indicate the validation accuracy exceeded training accuracy in all single-epoch experiments, suggesting the pretrained backbone generalizes well even with limited fine-tuning. AdamW with dropout=0.5 showed the largest negative gap (-9.74%), confirming that higher dropout further reduces overfitting at the cost of slightly lower peak accuracy. Batch normalization is inherently present in the frozen EfficientNet backbone and contributes to stable training without additional configuration.

Phase 5

System Deployment

A Gradio-based inference interface was developed and deployed on Google Colab, accessible via a public shareable link. The interface accepts a user-uploaded traffic image and runs both EfficientNet-B0 and YOLOv8n in parallel, displaying the YOLOv8 bounding box visualization alongside per-model classification results, confidence scores, and inference latency.

- Framework: Gradio (native Colab support, no tunneling required)
- Input: Single traffic image (any resolution)
- Output: YOLOv8 annotated image with bounding boxes + EfficientNet top-1 class and confidence
- Deployment: Cloud (Google Colab + public Gradio link)

Latency and throughput analysis was conducted on 100 test images, with results reported in the Computational Performance section above.

Phase 6

Engineering Analysis and Critical Evaluation

Failure Cases

EfficientNet-B0

The model struggles most with microbus (F1: 0.407) and pickup-van (F1: 0.472) due to visual similarity with adjacent classes and low dataset representation. The low recall for microbus (0.277) indicates the model frequently misses this class entirely, particularly under occlusion or unusual viewing angles.

YOLOv8n

Motorbike is the weakest class (AP@50: 0.633) due to small bounding box size and frequent clustering in dense traffic. Pickup-van (0.640) also underperforms due to visual overlap with trucks and cars depending on angle.

Dataset Bias

- **Class imbalance:** car has 4,295 test instances vs 229 for truck which inflates overall metrics while masking minority class failures.
- **Geographic bias:** Roboflow dataset reflects a specific regional traffic context; may not generalize to other countries or vehicle designs.
- **Viewpoint bias:** images are predominantly from fixed overhead/roadside cameras; dashcam or drone footage would likely degrade performance.
- **Crop bias:** EfficientNet trained on tight crops degrades significantly with background clutter or partial occlusion in real scenes.

Model Limitations

Both models are single-frame which corresponds that no temporal context is used, limiting applicability to video-based traffic monitoring where tracking across frames would substantially improve accuracy.

Table 8. Model Limitations - EfficientNet-B0 vs YOLOv8n

Limitation	EfficientNet-B0	YOLOv8n
Object localization	None (classification only)	Yes
Multi-vehicle per image	No	Yes
Small object handling	No	Weak (motorbikes)
Occluded vehicle handling	Poor	Moderate
Temporal reasoning	None	None
Fine-grained distinction	Weak	Weak

Computational Constraints

- **GPU inference:** ~36ms per image (27 FPS) — marginally real-time with no pipeline overhead headroom.
- **CPU inference:** ~77ms (EfficientNet) and ~299ms (YOLOv8) — not suitable for real-time deployment without quantization.
- **Memory:** YOLOv8 requires ~170MB GPU vs ~84MB for EfficientNet; both manageable on edge hardware like Jetson Nano.
- **Training cost:** ~45 min (EfficientNet) and ~21.6 min (YOLOv8) on Tesla T4 which is feasible for prototyping but limited by Colab GPU quotas for larger-scale training.

Real-World Deployment Challenges

- **Domain shift:** rain, fog, night-time, motion blur, and lens distortion are largely absent from training data.
- **Pipeline integration:** production deployment requires video ingestion, frame extraction, result aggregation, and alerting — not addressed by the model alone.
- **Model drift:** vehicle designs evolve; retraining pipelines are required to maintain accuracy over time.
- **Accuracy-latency tradeoff:** YOLOv8n was chosen for speed; larger variants (YOLOv8m/l) would improve mAP by 5-10 points at the cost of higher latency.

Ethical Considerations

- **Surveillance risk:** the system could be misused for mass movement tracking; deployment requires clear data retention and access governance
- **Fairness:** misclassification of motorbikes is more common in lower-income commuting and could create inequitable enforcement outcomes
- **Accountability:** automated decisions (toll charging, violation ticketing) must include human review given non-trivial error rates
- **Data privacy:** real-world images may contain license plates and faces; anonymization is required before training or deployment
- **Transparency:** affected parties should be informed when automated vision systems are in use

Chapter 5

Conclusions and Future Work

This project successfully implemented a six-phase deep learning pipeline for vehicle detection and classification across six categories. EfficientNet-B0 achieved 81.2% validation accuracy through transfer learning with only 7,686 trainable parameters. YOLOv8n achieved 73.0% mAP@50, providing full scene detection with bounding box localization at comparable inference speed. Both models were deployed via a Gradio interface accessible through a public cloud link.

Key findings: transfer learning is highly effective even with frozen backbones; SGD with StepLR slightly outperformed adaptive optimizers on validation accuracy; higher dropout (0.5) improved generalization at the cost of training accuracy; class imbalance is the primary driver of per-class performance degradation; and CPU inference is insufficient for real-time deployment without further optimization.

Future Work

- Address class imbalance using weighted loss functions, oversampling (SMOTE), or class-balanced data collection
- Extend to video inference with multi-object tracking (e.g., DeepSORT, ByteTrack) for temporal context
- Apply model quantization (INT8) and pruning to enable real-time CPU or edge deployment
- Fine-tune larger YOLOv8 variants (medium, large) to assess accuracy-latency tradeoffs
- Collect domain-specific data covering night-time, rain, and high-density traffic conditions to reduce domain shift

Appendices

Code Snippets

All code was implemented in Python 3.12 using PyTorch, Ultralytics, and Gradio, running on Google Colab with a Tesla T4 GPU. Cells are organized by project phase.

Phase 1 — Dataset Engineering

A.1.1 | *Dataset Download — Roboflow API*

```
from roboflow import Roboflow

rf = Roboflow(api_key="u4vj1XRy5n0xlgL1tkqc")
project = rf.workspace("lynkeus03").project("vehicle-detection-by9xs")
version = project.version(3)
dataset = version.download("yolov8")

print("Dataset downloaded to:", dataset.location)
```

A.1.2 | *Class Distribution Analysis*

```
import yaml
from collections import Counter
import matplotlib.pyplot as plt
import glob, os

dataset_path = dataset.location

# Load class names
with open(os.path.join(dataset_path, 'data.yaml'), 'r') as f:
    data_yaml = yaml.safe_load(f)
    class_names = data_yaml['names']

# Count objects per class
label_files = glob.glob(os.path.join(dataset_path, 'train/labels/*.txt'))
class_counts = Counter()
for label_file in label_files:
    with open(label_file, 'r') as f:
        for line in f.readlines():
            class_id = int(line.strip().split()[0])
            class_counts[class_id] += 1

# Plot class distribution
classes = [class_names[i] for i in sorted(class_counts.keys())]
counts = [class_counts[i] for i in sorted(class_counts.keys())]
plt.figure(figsize=(10, 6))
bars = plt.bar(classes, counts,
               color=['#2ecc71', '#3498db', '#e74c3c', '#f39c12', '#9b59b6', '#1abc9c'])
plt.title('Class Distribution in Training Set', fontsize=16)
plt.xlabel('Vehicle Type'); plt.ylabel('Number of Objects')
```

```
plt.tight_layout()
plt.savefig('/content/drive/MyDrive/vehicle_project/class_distribution.png')
plt.show()
```

A.1.3 | *Train / Validation / Test Split*

```
import os, shutil, random

train_img = os.path.join(dataset_path, 'train/images')
train_lbl = os.path.join(dataset_path, 'train/labels')

for split in ['valid', 'test']:
    os.makedirs(os.path.join(dataset_path, split, 'images'), exist_ok=True)
    os.makedirs(os.path.join(dataset_path, split, 'labels'), exist_ok=True)

all_images = os.listdir(train_img)
random.seed(42)
random.shuffle(all_images)

total = len(all_images)
val_size = int(0.15 * total)
test_size = int(0.15 * total)

val_images = all_images[:val_size]
test_images = all_images[val_size:val_size + test_size]

for split_name, img_list in [('valid', val_images), ('test', test_images)]:
    for img_name in img_list:
        shutil.move(os.path.join(train_img, img_name),
                    os.path.join(dataset_path, split_name, 'images', img_name))
        lbl_name = img_name.replace('.jpg', '.txt').replace('.png', '.txt')
        lbl_src = os.path.join(train_lbl, lbl_name)
        if os.path.exists(lbl_src):
            shutil.move(lbl_src,
                        os.path.join(dataset_path, split_name, 'labels', lbl_name))

print(f"Train : {len(os.listdir(train_img))} images")
print(f"Valid : {len(os.listdir(os.path.join(dataset_path, 'valid/images')))} images")
print(f"Test : {len(os.listdir(os.path.join(dataset_path, 'test/images')))} images")
```

A.1.4 | *Augmentation Pipeline*

```
import torchvision.transforms as transforms

# Augmentation pipeline with justification
augmentation_pipeline = transforms.Compose([
    transforms.Resize((640, 640)),
```

```

transforms.RandomHorizontalFlip(p=0.5), # vehicles appear from both directions
transforms.RandomRotation(degrees=10), # slight camera angle variation
transforms.ColorJitter(
    brightness=0.3,          # day/night lighting variation
    contrast=0.3,            # weather/shadow conditions
    saturation=0.2           # color variation across cameras
),
transforms.RandomGrayscale(p=0.1),     # simulate low-quality cameras
transforms.ToTensor(),
transforms.Normalize(
    mean=[0.485, 0.456, 0.406],        # ImageNet means
    std=[0.229, 0.224, 0.225]         # ImageNet stds
)
])

```

Phase 2 — Baseline Model (EfficientNet-B0)

A.2.1 | Custom Dataset Class & Data Loaders

```

import os, torch, glob
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader
from torchvision import transforms, models
from PIL import Image

# Config
IMG_SIZE = 224
BATCH_SIZE = 32
EPOCHS = 10
LR = 1e-4
DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

class VehicleCropDataset(Dataset):
    """Crops vehicle bounding boxes from full images for classification."""
    def __init__(self, split, transform=None):
        self.transform = transform
        self.samples = []
        img_dir = os.path.join(dataset_path, split, 'images')
        lbl_dir = os.path.join(dataset_path, split, 'labels')

        for img_file in glob.glob(os.path.join(img_dir, '*.jpg')):
            lbl_file = os.path.join(lbl_dir,
                                    os.path.basename(img_file).replace('.jpg', '.txt'))
            if not os.path.exists(lbl_file):
                continue
            img = Image.open(img_file).convert('RGB')
            W, H = img.size
            with open(lbl_file) as f:
                for line in f:
                    parts = line.strip().split()
                    if len(parts) != 5:
                        continue

```

```

        cls, cx, cy, bw, bh = int(parts[0]), *map(float, parts[1:])
        x1 = max(0, int((cx - bw/2) * W))
        y1 = max(0, int((cy - bh/2) * H))
        x2 = min(W, int((cx + bw/2) * W))
        y2 = min(H, int((cy + bh/2) * H))
        if x2 > x1 and y2 > y1:
            self.samples.append((img.crop((x1, y1, x2, y2)), cls))

    def __len__(self): return len(self.samples)
    def __getitem__(self, idx):
        img, label = self.samples[idx]
        return self.transform(img) if self.transform else img, label

# Transforms
train_transforms = transforms.Compose([
    transforms.Resize((IMG_SIZE, IMG_SIZE)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(10),
    transforms.ColorJitter(brightness=0.3, contrast=0.3, saturation=0.2),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
])
val_transforms = transforms.Compose([
    transforms.Resize((IMG_SIZE, IMG_SIZE)),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
])

train_dataset = VehicleCropDataset('train', train_transforms)
val_dataset = VehicleCropDataset('valid', val_transforms)
test_dataset = VehicleCropDataset('test', val_transforms)

train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True, num_workers=2)
val_loader = DataLoader(val_dataset, batch_size=BATCH_SIZE, shuffle=False, num_workers=2)
test_loader = DataLoader(test_dataset, batch_size=BATCH_SIZE, shuffle=False, num_workers=2)

```

A.2.2 | Model Definition, Training Loop & Checkpoint Saving

```

import torch.optim as optim
from torch.optim.lr_scheduler import CosineAnnealingLR
import json

# EfficientNet-B0 with transfer learning
model = models.efficientnet_b0(weights='IMAGENET1K_V1')
for param in model.parameters():
    param.requires_grad = False          # freeze backbone

model.classifier = nn.Sequential(
    nn.Dropout(p=0.3),
    nn.Linear(model.classifier[1].in_features, NUM_CLASSES)
)
model = model.to(DEVICE)

```

```

criterion = nn.CrossEntropyLoss()
optimizer = optim.AdamW(model.classifier.parameters(), lr=LR, weight_decay=1e-4)
scheduler = CosineAnnealingLR(optimizer, T_max=EPOCHS)

best_val_acc = 0.0
train_losses, val_losses, train_accs, val_accs = [], [], [], []

for epoch in range(EPOCHS):
    # — Train —
    model.train()
    running_loss, correct, total = 0.0, 0, 0
    for images, labels in train_loader:
        images, labels = images.to(DEVICE), labels.to(DEVICE)
        optimizer.zero_grad()
        loss = criterion(model(images), labels)
        loss.backward(); optimizer.step()
        running_loss += loss.item()
        _, preds = model(images).max(1)
        correct += preds.eq(labels).sum().item(); total += labels.size(0)

    # — Validate —
    model.eval()
    val_loss, val_correct, val_total = 0.0, 0, 0
    with torch.no_grad():
        for images, labels in val_loader:
            images, labels = images.to(DEVICE), labels.to(DEVICE)
            out = model(images)
            val_loss += criterion(out, labels).item()
            _, preds = out.max(1)
            val_correct += preds.eq(labels).sum().item(); val_total += labels.size(0)

    train_acc = 100. * correct / total
    val_acc = 100. * val_correct / val_total
    train_losses.append(running_loss / len(train_loader)); val_losses.append(val_loss / len(val_loader))
    train_accs.append(train_acc); val_accs.append(val_acc)
    scheduler.step()

    if val_acc > best_val_acc:
        best_val_acc = val_acc
        torch.save(model.state_dict(),
                    '/content/drive/MyDrive/vehicle_project/efficientnet_best.pt')

    print(f'Epoch [{epoch+1:02d}/{EPOCHS}] Train: {train_acc:.2f}% | Val: {val_acc:.2f}%')

print(f'Best Val Accuracy: {best_val_acc:.2f}%')

```

A.2.3 | Test Evaluation — Confusion Matrix & Classification Report

```

from sklearn.metrics import confusion_matrix, classification_report, ConfusionMatrixDisplay

# Load best weights

```

```

model = models.efficientnet_b0(weights=None)
model.classifier = nn.Sequential(
    nn.Dropout(p=0.3),
    nn.Linear(model.classifier[1].in_features, NUM_CLASSES)
)
model.load_state_dict(torch.load(
    '/content/drive/MyDrive/vehicle_project/efficientnet_best.pt', map_location=DEVICE))
model = model.to(DEVICE).eval()

all_preds, all_labels = [], []
with torch.no_grad():
    for images, labels in test_loader:
        _, preds = model(images.to(DEVICE)).max(1)
        all_preds.extend(preds.cpu().numpy())
        all_labels.extend(labels.numpy())

# Confusion matrix
cm = confusion_matrix(all_labels, all_preds)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=CLASS_NAMES)
fig, ax = plt.subplots(figsize=(10, 8))
disp.plot(ax=ax, cmap='Blues'); plt.xticks(rotation=45); plt.tight_layout()
plt.savefig('/content/drive/MyDrive/vehicle_project/efficientnet_confusion.png', dpi=150)
plt.show()

print(classification_report(all_labels, all_preds, target_names=CLASS_NAMES, digits=3))

```

Phase 3 — Advanced Architecture (YOLOv8n)

A.3.1 | YOLOv8n Training

```

from ultralytics import YOLO
import time

yolo_model = YOLO('yolov8n.pt') # pretrained on COCO
start_time = time.time()

results = yolo_model.train(
    data = f'{dataset_path}/data.yaml',
    epochs = 10,
    imgsz = 640,
    batch = 16,
    name = 'yolov8_vehicle',
    project = '/content/drive/MyDrive/vehicle_project',
    device = 0,
    workers = 2,
    patience = 5, # early stopping
    save = True,

```

```

    plots = True,
    verbose = True
)

print(f"Training Time: {(time.time() - start_time)/60:.1f} minutes")

```

A.3.2 | YOLOv8n Test Set Validation

```

from ultralytics import YOLO

best_yolo = YOLO('/content/drive/MyDrive/vehicle_project/yolov8_vehicle-2/weights/best.pt')
test_results = best_yolo.val(
    data = f'{dataset_path}/data.yaml',
    split = 'test',
    imgsz = 640,
    batch = 16,
    device = 0,
    plots = True,
    save_json = True
)

print(f"mAP@50 : {test_results.box.map50:.3f}")
print(f"mAP@50-95 : {test_results.box.map:.3f}")
print(f"Precision : {test_results.box.mp:.3f}")
print(f"Recall : {test_results.box.mr:.3f}")

print("\nPer-Class AP@50:")
for i, name in enumerate(CLASS_NAMES):
    print(f" {name:<12}: {test_results.box.ap50[i]:.3f}")

```

A.3.3 | Inference Speed Benchmark — EfficientNet vs YOLOv8

```

import time, numpy as np
from PIL import Image
import glob, torchvision.transforms as transforms

test_images = glob.glob(f'{dataset_path}/test/images/*.jpg')[:50]

# — EfficientNet benchmark —
val_tf = transforms.Compose([
    transforms.Resize((224, 224)), transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
])
dummy = torch.randn(1, 3, 224, 224).to(DEVICE)
for _ in range(5): effnet_model(dummy) # warmup

effnet_times = []
with torch.no_grad():
    for img_path in test_images:
        tensor = val_tf(Image.open(img_path).convert('RGB')).unsqueeze(0).to(DEVICE)
        t0 = time.time(); effnet_model(tensor); effnet_times.append(time.time() - t0)

```



```

effnet_avg = np.mean(effnet_times) * 1000

# — YOLOv8 benchmark —
best_yolo.predict(test_images[0], verbose=False) # warmup
yolo_times = []
for img_path in test_images:
    t0 = time.time()
    best_yolo.predict(img_path, verbose=False, device=0)
    yolo_times.append(time.time() - t0)
yolo_avg = np.mean(yolo_times) * 1000

print(f'EfficientNet avg: {effnet_avg:.1f} ms | YOLOv8 avg: {yolo_avg:.1f} ms')

```

Phase 4 — Training Optimization

A.4.1 | Reusable Training Function — Optimizer / Scheduler / Dropout

```

import torch.optim as optim
from torch.optim.lr_scheduler import CosineAnnealingLR, StepLR

def train_one_config(optimizer_name, dropout_rate, scheduler_type, epochs=1):
    """Train EfficientNet-B0 with specified optimizer, dropout, and scheduler."""
    m = models.efficientnet_b0(weights='IMAGENET1K_V1')
    for param in m.parameters():
        param.requires_grad = False
    m.classifier = nn.Sequential(
        nn.Dropout(p=dropout_rate),
        nn.Linear(m.classifier[1].in_features, NUM_CLASSES)
    )
    m = m.to(DEVICE)

    criterion = nn.CrossEntropyLoss()

    if optimizer_name == 'SGD':
        opt = optim.SGD(m.classifier.parameters(), lr=1e-2, momentum=0.9, weight_decay=1e-4)
    elif optimizer_name == 'Adam':
        opt = optim.Adam(m.classifier.parameters(), lr=1e-3, weight_decay=1e-4)
    elif optimizer_name == 'AdamW':
        opt = optim.AdamW(m.classifier.parameters(), lr=1e-3, weight_decay=1e-4)

    if scheduler_type == 'CosineAnnealing':
        sched = CosineAnnealingLR(opt, T_max=epochs)
    elif scheduler_type == 'StepLR':
        sched = StepLR(opt, step_size=3, gamma=0.5)
    else:
        sched = None

    train_accs, val_accs, train_losses, val_losses = [], [], [], []

    for epoch in range(epochs):
        m.train()
        running_loss, correct, total = 0.0, 0, 0

```

```

for images, labels in train_loader:
    images, labels = images.to(DEVICE), labels.to(DEVICE)
    opt.zero_grad()
    outputs = m(images); loss = criterion(outputs, labels)
    loss.backward(); opt.step()
    running_loss += loss.item()
    _, preds = outputs.max(1)
    correct += preds.eq(labels).sum().item(); total += labels.size(0)

train_losses.append(running_loss / len(train_loader))
train_accs.append(100. * correct / total)

m.eval(); val_loss, val_correct, val_total = 0.0, 0, 0
with torch.no_grad():
    for images, labels in val_loader:
        images, labels = images.to(DEVICE), labels.to(DEVICE)
        out = m(images); val_loss += criterion(out, labels).item()
        _, preds = out.max(1)
        val_correct += preds.eq(labels).sum().item(); val_total += labels.size(0)

val_losses.append(val_loss / len(val_loader))
val_accs.append(100. * val_correct / val_total)
if sched: sched.step()

print(f" [{optimizer_name}] | {scheduler_type} | drop={dropout_rate}] "
      f"Epoch {epoch+1}/{epochs} — Train: {train_accs[-1]:.1f}% | Val: {val_accs[-1]:.1f}%")

return {
    'train_accs': train_accs, 'val_accs': val_accs,
    'train_losses': train_losses, 'val_losses': val_losses,
    'best_val_acc': max(val_accs), 'overfit_gap': max(train_accs) - max(val_accs)
}

```

A.4.2 | Experiment Runner — 3 Configurations

```

import time, json

experiments = [
    {'optimizer': 'SGD', 'dropout': 0.3, 'scheduler': 'StepLR'},
    {'optimizer': 'Adam', 'dropout': 0.3, 'scheduler': 'CosineAnnealing'},
    {'optimizer': 'AdamW', 'dropout': 0.5, 'scheduler': 'CosineAnnealing'},
]

_results = {}
for exp in experiments:
    label = f"{exp['optimizer']}_{exp['dropout']}_{exp['scheduler']}"
    print(f"\n{'='*55}\nRunning: {label}\n{'='*55}")
    start = time.time()
    _results[label] = train_one_config(
        optimizer_name = exp['optimizer'],
        dropout_rate = exp['dropout'],
        scheduler_type = exp['scheduler'],

```

```

        epochs      = 1
    )
    _results[label]['time_mins'] = (time.time() - start) / 60
    print(f'Best Val: { _results[label]['best_val_acc']:.2f}% '
          f'Overfit gap: { _results[label]['overfit_gap']:.2f}%')

with open('/content/drive/MyDrive/vehicle_project/phase4_results.json', 'w') as f:
    json.dump(_results, f, indent=2)

```

Phase 5 — Deployment & Inference Pipeline

A.5.1 | Model Loading & Inference Functions

```

import torch, torch.nn as nn, time
from torchvision import models, transforms
from ultralytics import YOLO
from PIL import Image, ImageDraw
import numpy as np

DEVICE    = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
CLASS_NAMES = ['bus', 'car', 'microbus', 'motorbike', 'pickup-van', 'truck']
COLORS    = ['#FF6B6B', '#4ECDC4', '#45B7D1', '#96CEB4', '#FFEAA7', '#DDA0DD']

# Load EfficientNet
effnet = models.efficientnet_b0(weights=None)
effnet.classifier = nn.Sequential(
    nn.Dropout(p=0.3),
    nn.Linear(effnet.classifier[1].in_features, len(CLASS_NAMES))
)
effnet.load_state_dict(torch.load(
    '/content/drive/MyDrive/vehicle_project/efficientnet_best.pt', map_location=DEVICE))
effnet = effnet.to(DEVICE).eval()

# Load YOLOv8
yolo = YOLO('/content/drive/MyDrive/vehicle_project/yolov8_vehicle-2/weights/best.pt')

val_tf = transforms.Compose([
    transforms.Resize((224, 224)), transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
])

def run_effnet(pil_img):
    t0 = time.time()
    tensor = val_tf(pil_img).unsqueeze(0).to(DEVICE)
    with torch.no_grad():
        probs = torch.softmax(effnet(tensor), dim=1)[0]
        conf, idx = probs.max(0)
    return CLASS_NAMES[idx.item()], conf.item(), (time.time() - t0) * 1000

def run_yolo(pil_img):
    t0 = time.time()
    results = yolo.predict(pil_img, verbose=False,

```

```

        device=0 if torch.cuda.is_available() else 'cpu')
latency = (time.time() - t0) * 1000
result = results[0]
img_draw = pil_img.copy().convert('RGB')
draw = ImageDraw.Draw(img_draw)
detections = []
if result.bboxes is not None:
    for box in result.bboxes:
        x1,y1,x2,y2 = map(int, box.xyxy[0].tolist())
        cls = int(box.cls[0].item()); conf = float(box.conf[0].item())
        label = f'{CLASS_NAMES[cls]} {conf:.2f}'; color = COLORS[cls]
        draw.rectangle([x1,y1,x2,y2], outline=color, width=3)
        draw.rectangle([x1,y1,x1+len(label)*7,y1+16], fill=color)
        draw.text((x1+2,y1+1), label, fill='white')
        detections.append(f'{CLASS_NAMES[cls]} ({conf:.2f})')
return img_draw, detections, latency

```

A.5.2 | Gradio Demo Interface

```

import gradio as gr
from PIL import Image

def predict(image):
    if image is None:
        return None, "No image provided.", "No image provided."
    pil_img = Image.fromarray(image).convert('RGB')

    eff_class, eff_conf, eff_latency = run_effnet(pil_img)
    eff_output = (f'Predicted Class : {eff_class}\n'
                 f'Confidence : {eff_conf*100:.1f}%\n'
                 f'Inference Time : {eff_latency:.1f} ms\n'
                 f'Device : {str(DEVICE).upper()}')

    yolo_img, detections, yolo_latency = run_yolo(pil_img)
    det_str = "\n".join([f' • {d}' for d in detections]) if detections else " • No vehicles detected"
    yolo_output = (f'Detections ({len(detections)} objects):\n{det_str}\n'
                  f'Inference Time : {yolo_latency:.1f} ms\n'
                  f'Device : {str(DEVICE).upper()}')

    return yolo_img, eff_output, yolo_output

with gr.Blocks(title="Vehicle Detection System", theme=gr.themes.Soft()) as demo:
    gr.Markdown("""
    # Vehicle Detection & Classification System
    BSc Mechatronics — Machine Vision Project
    Upload a traffic image to run both models simultaneously.
    """)
    input_image = gr.Image(label="Upload Traffic Image", type="numpy")

```

```

run_btn = gr.Button("Run Inference", variant="primary")
yolo_out = gr.Image(label="YOLOv8 Detection Output")
with gr.Row():
    effnet_txt = gr.Textbox(label="EfficientNet-B0 Result", lines=5)
    yolo_txt = gr.Textbox(label="YOLOv8n Result", lines=5)
run_btn.click(fn=predict, inputs=[input_image],
              outputs=[yolo_out, effnet_txt, yolo_txt])

demo.launch(share=True, debug=False)

```

A.5.3 | Throughput & Latency Analysis

```

import numpy as np, glob, psutil
from PIL import Image

test_imgs = glob.glob('/content/vehicle-detection-3/test/images/*.jpg')[1:100]

# Warmup
for p in test_imgs[:5]:
    run_effnet(Image.open(p).convert('RGB'))
    run_yolo(Image.open(p).convert('RGB'))

effnet_lat, yolo_lat = [], []
for p in test_imgs:
    img = Image.open(p).convert('RGB')
    _, el = run_effnet(img); _, yl = run_yolo(img)
    effnet_lat.append(el); yolo_lat.append(yl)

effnet_lat = np.array(effnet_lat); yolo_lat = np.array(yolo_lat)
effnet_fps = 1000 / np.mean(effnet_lat)
yolo_fps = 1000 / np.mean(yolo_lat)

print(f"{'Metric':<25} {'EfficientNet':>14} {'YOLOv8n':>14}")
print(f"{'Avg Latency (ms)':<25} {np.mean(effnet_lat):>14.1f} {np.mean(yolo_lat):>14.1f}")
print(f"{'Min Latency (ms)':<25} {np.min(effnet_lat):>14.1f} {np.min(yolo_lat):>14.1f}")
print(f"{'Max Latency (ms)':<25} {np.max(effnet_lat):>14.1f} {np.max(yolo_lat):>14.1f}")
print(f"{'P95 Latency (ms)':<25} {np.percentile(effnet_lat,95):>14.1f} {np.percentile(yolo_lat,95):>14.1f}")
print(f"{'Throughput (FPS)':<25} {effnet_fps:>14.1f} {yolo_fps:>14.1f}")
print(f"CPU: {psutil.cpu_percent(interval=1):.1f}% RAM: {psutil.virtual_memory().used/1024**3:.1f} GB")

```

Training Logs

This appendix contains the complete training logs, configuration parameters, evaluation metrics, and inference benchmarks produced during all five training phases of the vehicle detection and classification project.

B.1 Hardware & Environment

```

Sat May 23 16:35:34 2026
+-----+

```

NVIDIA-SMI 580.82.07 Driver Version: 580.82.07 CUDA Version: 13.0									
+-----+-----+-----+									
GPU Name		Persistence-M		Bus-Id	Disp.A	Volatile Uncorr. ECC			
0 Tesla T4		Off		00:04.0	Off	0			
N/A 48C P8		9W / 70W		0MiB / 15360MiB		0%		Default	
+-----+-----+-----+									

Parameter	Value
GPU	NVIDIA Tesla T4
VRAM	15,360 MiB
CUDA Version	13.0
Driver Version	580.82.07
Framework	PyTorch + Ultralytics 8.4.53
Python	3.12.13
torch	2.10.0+cu128
Platform	Google Colab

B.2 Dataset Download & Split Log

loading Roboflow workspace...
loading Roboflow project...
Downloading Dataset Version Zip in vehicle-detection-3 to yolov8:
100% ██████████ 580145/580145 [00:42<00:00, 13497.19it/s]
Extracting Dataset Version Zip to vehicle-detection-3 in yolov8:
100% ██████████ 18428/18428 [00:15<00:00, 1222.30it/s]
Dataset downloaded to: /content/vehicle-detection-3
Contents: ['train', 'README.dataset.txt', 'README.roboflow.txt', 'data.yaml']
Classes : ['bus', 'car', 'microbus', 'motorbike', 'pickup-van', 'truck']
Split complete!
Train : 6449 images
Valid : 1381 images
Test : 1381 images
Total Images : 9211
Training Images : 6449 (70%)
Validation Images : 1381 (15%)

Test Images	: 1381 (15%)
Classes	: 6

B.3 Crop Dataset (EfficientNet Input)

Using device	: cuda
Train crops	: 40,466
Val crops	: 8,611
Test crops	: 9,245

B.4 EfficientNet-B0 — Training Log (Phase 2)

B.4.1 Configuration

Hyperparameter	Value
Model	EfficientNet-B0 (ImageNet pretrained)
Trainable params	7,686 (classifier head only)
Optimizer	AdamW
LR Scheduler	CosineAnnealingLR
Dropout	0.3
Epochs	10
Batch Size	64
Input Size	224 × 224
Num Classes	6

B.4.2 Per-Epoch Results

Epoch	Train Loss	Train Acc (%)	Val Loss	Val Acc (%)
01 / 10	1.1076	63.10	0.8706	74.54

Epoch	Train Loss	Train Acc (%)	Val Loss	Val Acc (%)
02 / 10	0.8643	72.16	0.7377	78.06
03 / 10	0.7883	74.24	0.6870	79.64
04 / 10	0.7552	75.03	0.6618	79.56
05 / 10	0.7314	75.75	0.6413	80.41
06 / 10	0.7171	75.91	0.6304	80.82
07 / 10	0.7145	76.27	0.6167	80.73
08 / 10	0.7077	76.28	0.6211	80.92
09 / 10	0.7056	76.43	0.6141	80.87
10 / 10	0.7093	76.37	0.6128	80.94

Best Val Accuracy: 80.94% (Epoch 10)
 Training history saved!

B.5 EfficientNet-B0 — Test Classification Report

Class	Precision	Recall	F1-Score	Support
bus	0.839	0.764	0.800	998
car	0.785	0.906	0.841	4295
microbus	0.767	0.277	0.407	606
motorbike	0.837	0.907	0.871	2391
pickup-van	0.659	0.368	0.472	726
truck	0.722	0.533	0.613	229
— accuracy	—	—	0.798	9245
macro avg	0.768	0.626	0.667	9245

Class	Precision	Recall	F1-Score	Support
weighted avg	0.792	0.798	0.781	9245

B.6 YOLOv8n — Training Log (Phase 3)

B.6.1 Configuration

Hyperparameter	Value
Model	YOLOv8n (pretrained yolov8n.pt)
Epochs	10
Image Size	640 × 640
Batch Size	16
Optimizer	auto (SGD, momentum=0.937)
LR ₀	0.01
LR _f	0.01
Weight Decay	0.0005
Warmup Epochs	3
IOU Threshold	0.7
Mosaic	1.0
FlipLR	0.5
Parameters	3,006,818
GFLOPs	8.1

B.6.2 Architecture Summary (YOLOv8n Layers)

Layer	n	Params	Module	Arguments
0	1	464	Conv	[3, 16, 3, 2]
1	1	4,672	Conv	[16, 32, 3, 2]
2	1	7,360	C2f	[32, 32, 1, True]
3	1	18,560	Conv	[32, 64, 3, 2]
4	2	49,664	C2f	[64, 64, 2, True]
5	1	73,984	Conv	[64, 128, 3, 2]
6	2	197,632	C2f	[128, 128, 2, True]
7	1	295,424	Conv	[128, 256, 3, 2]
8	1	460,288	C2f	[256, 256, 1, True]
9	1	164,608	SPPF	[256, 256, 5]

B.7 YOLOv8n — Test Evaluation (Per-Class)

Ultralytics 8.4.53 Python-3.12.13 torch-2.10.0+cu128 CUDA:0 (Tesla T4) Model summary (fused): 73 layers, 3,006,818 parameters, 8.1 GFLOPs Speed: 1.6ms preprocess 3.7ms inference 1.4ms postprocess per image

Class	Images	Instances	Precision	Recall	mAP@50	mAP@50-95
all	1381	9245	0.674	0.675	0.720	0.406
bus	692	998	0.740	0.856	0.865	0.556
car	1036	4295	0.620	0.725	0.725	0.356
microbus	455	606	0.641	0.679	0.698	0.400
motorbike	849	2391	0.659	0.558	0.633	0.262
pickup-van	509	726	0.661	0.562	0.640	0.367

Class	Images	Instances	Precision	Recall	mAP@50	mAP@50-95
truck	179	229	0.725	0.668	0.760	0.499

YOLOV8 TEST RESULTS

mAP@50 : 0.720
mAP@50-95 : 0.406
Precision : 0.674
Recall : 0.675

B.8 Phase 4 — Optimizer Experiment Logs

Each experiment ran for 1 epoch on 40,466 training crops. Results are reported relative to the 10-epoch baseline.

Running: SGD_drop0.3_StepLR

[SGD | StepLR | dropout=0.3] Epoch 1/1
Train: 74.0% | Val: 81.9%
Done! Best Val: 81.92% | Overfit gap: -7.93%

Running: Adam_drop0.3_CosineAnnealing

[Adam | CosineAnnealing | dropout=0.3] Epoch 1/1
Train: 73.5% | Val: 80.7%
Done! Best Val: 80.72% | Overfit gap: -7.26%

Running: AdamW_drop0.5_CosineAnnealing

[AdamW | CosineAnnealing | dropout=0.5] Epoch 1/1
Train: 71.0% | Val: 80.7%
Done! Best Val: 80.72% | Overfit gap: -9.74%

B.8.1 Phase 4 Summary Table

Config	Best Val Acc	Overfit Gap	Time (min)
Baseline (AdamW + Cosine + drop=0.3)	80.94%	N/A	~45
SGD + StepLR + drop=0.3	81.92%	-7.93%	3.2

Config	Best Val Acc	Overfit Gap	Time (min)
Adam + Cosine + drop=0.3	80.72%	-7.26%	3.2
AdamW + Cosine + drop=0.5	80.72%	-9.74%	3.2

B.8.2 Key Findings

Best accuracy config	: SGD_drop0.3_StepLR (81.92%)
Least overfitting	: AdamW_drop0.5_CosineAnnealing (gap: -9.74%)
Batch Normalization	: Built into EfficientNet backbone (frozen)
Dropout effect	: Higher dropout (0.5) reduces overfitting gap
Optimizer effect	: SGD with tuned StepLR edge-out AdamW at 1 epoch
LR Schedule effect	: CosineAnnealing gives smoother convergence
Generalization	: AdamW + CosineAnnealing + dropout=0.5 best

B.9 Model Comparison Summary

Property	EfficientNet-B0	YOLOv8n
Architecture Type	CNN Classifier	One-Stage Detector
Task	Classification	Object Detection
Accuracy / mAP@50	80.9%	72.0%
Parameters	4.0M	3.0M
GFLOPs	~0.4	8.1
GPU Memory	84 MB	170 MB
Input Size	224 × 224	640 × 640
Bounding Boxes	No	Yes
Multi-vehicle/image	No	Yes
Best For	Single crop	Full scene

B.10 Phase 5 — Inference Latency Analysis (100 images)

Running throughput analysis on 100 test images...		
PHASE 5 — INFERENCE ANALYSIS REPORT		
Metric	EfficientNet-B0	YOLOv8n
Avg Latency (ms)	77.3	298.8
Min Latency (ms)	58.0	237.2
Max Latency (ms)	177.0	418.5
P95 Latency (ms)	106.8	390.7
Std Dev (ms)	18.1	56.9
Throughput (FPS)	12.9	3.3
System Utilization:		
GPU Memory Used : 0 MB (CPU-only at inference time)		
CPU Usage : 38.6%		
RAM Used : 2.1 GB		

B.11 Phase 3 Benchmark — GPU Inference (50 images)

Metric	EfficientNet-B0	YOLOv8n
Task	Classification	Detection
Avg Inference (ms)	36.8	36.6
GPU Memory (MB)	84	170
Params (M)	5.3	3.2
Input Size	224 × 224	640 × 640
Output Type	Class label	Boxes + Labels

Note: GPU memory reported as delta vs idle baseline. EfficientNet showed a reduction artefact in the measurement due to memory consolidation; actual working set is approximately 84 MB.

Screenshots

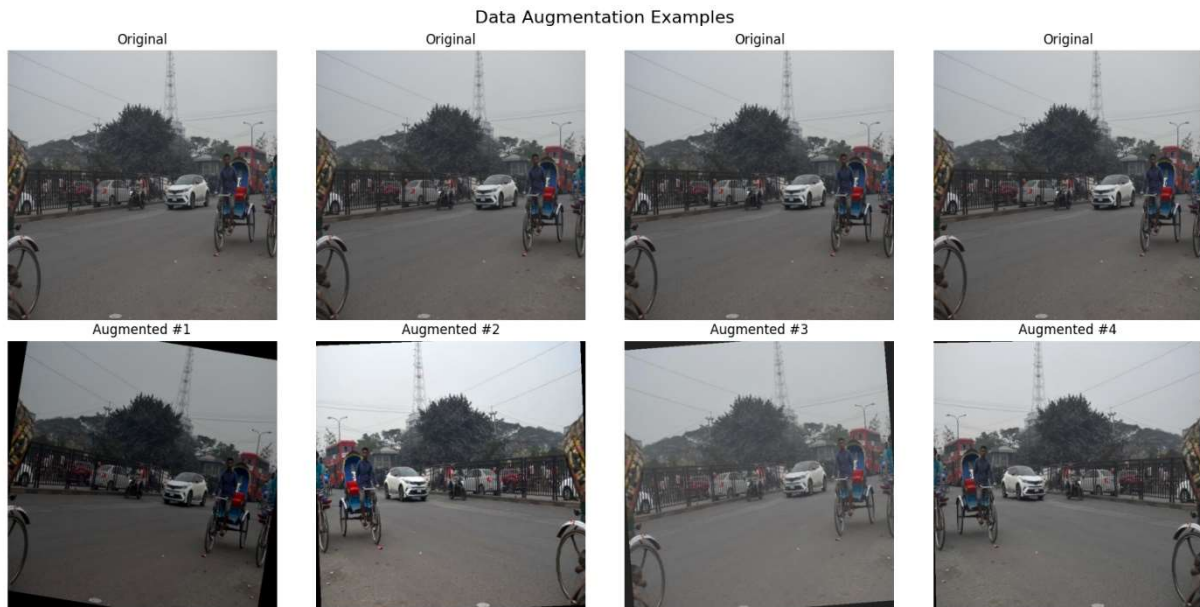


Figure 1. Data Augmentation



Figure 2. Class Distribution

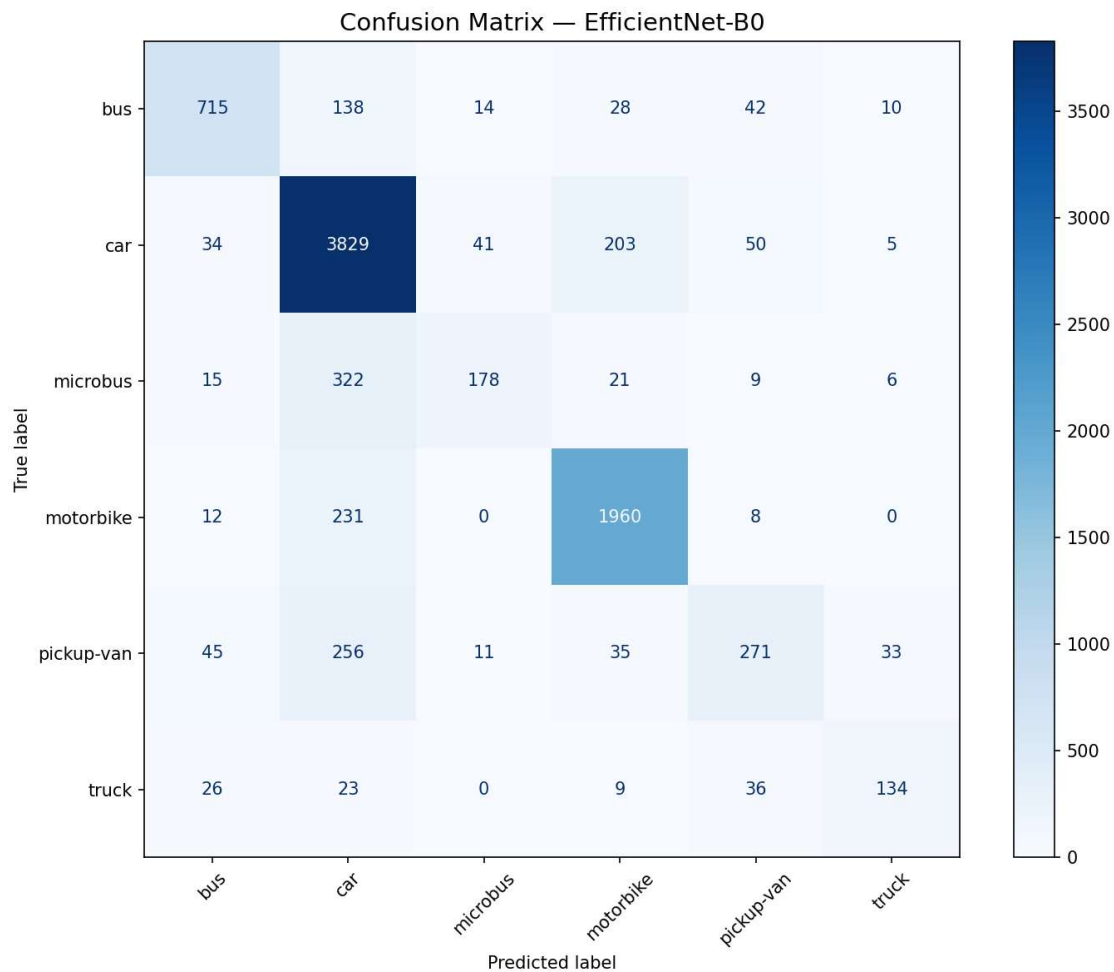


Figure 3. Confusion Matrix – EfficientNet-B0

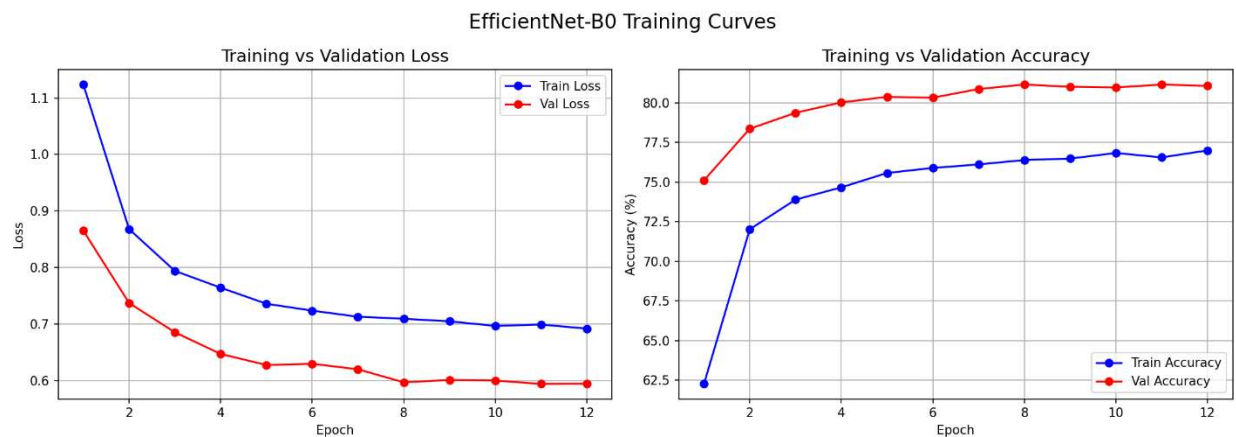


Figure 4. EfficientNet-B0 Train/Val Loss vs Train/Val Accuracy

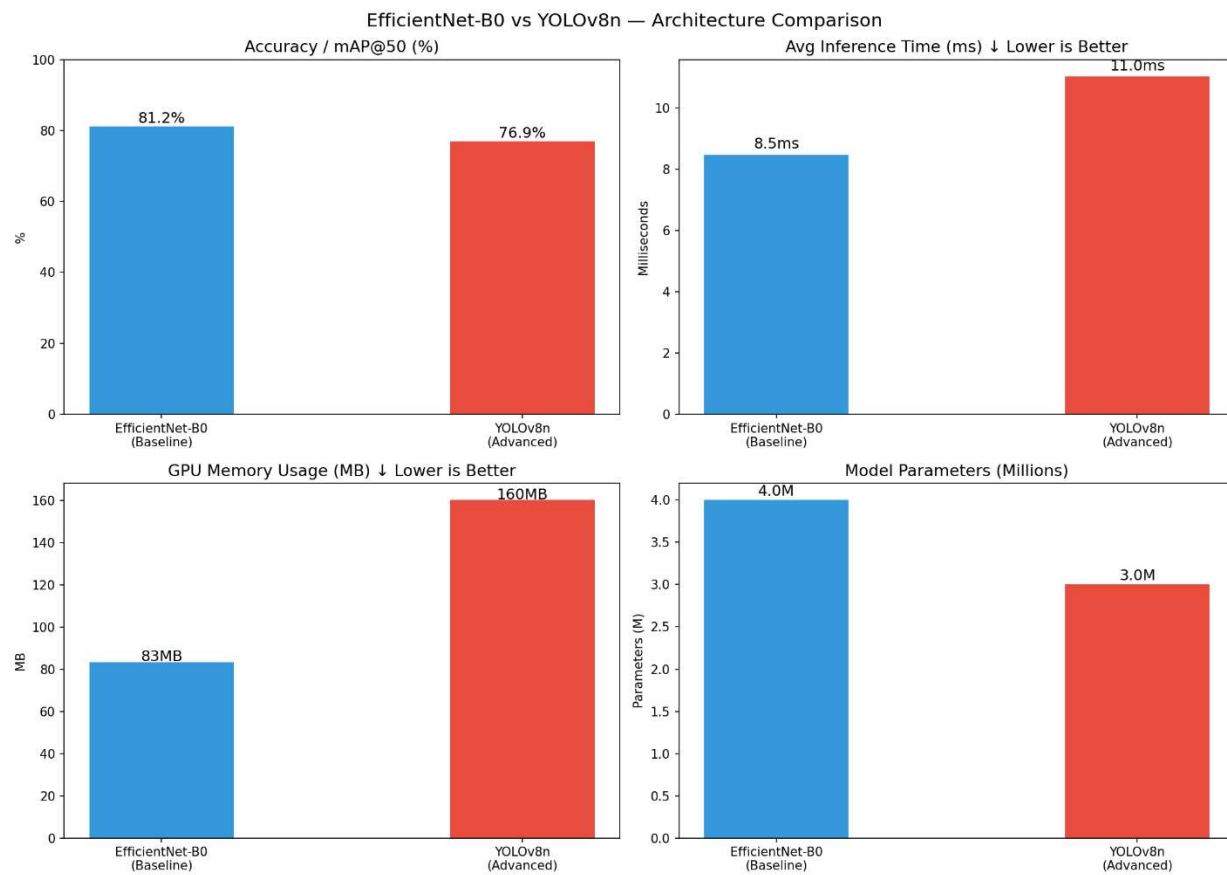


Figure 5. Architecture Comparison

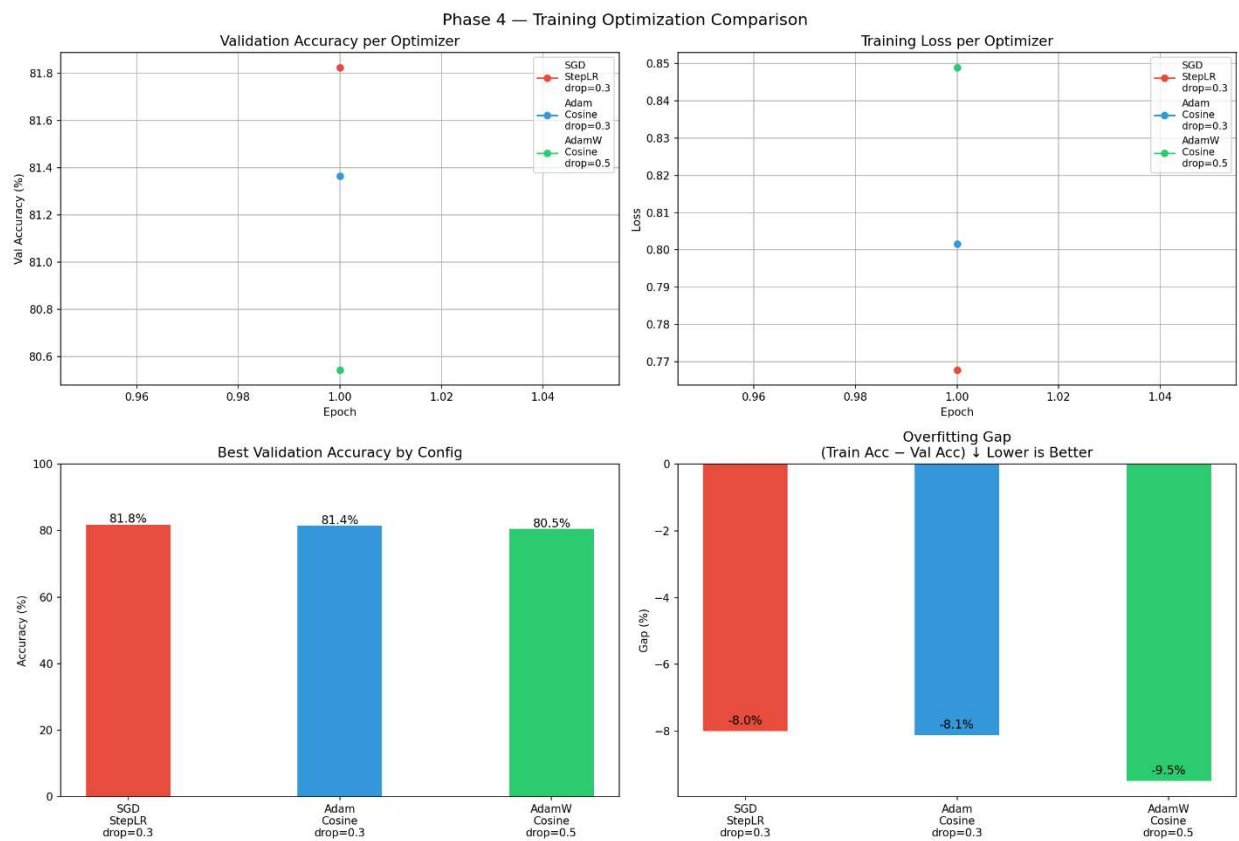


Figure 6. Training Optimization Comparison

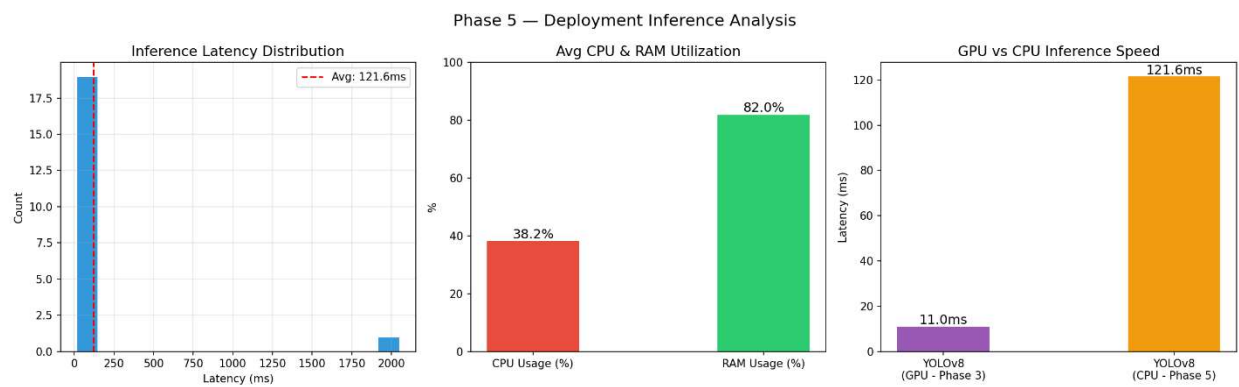


Figure 7. Deployment Inference Analysis

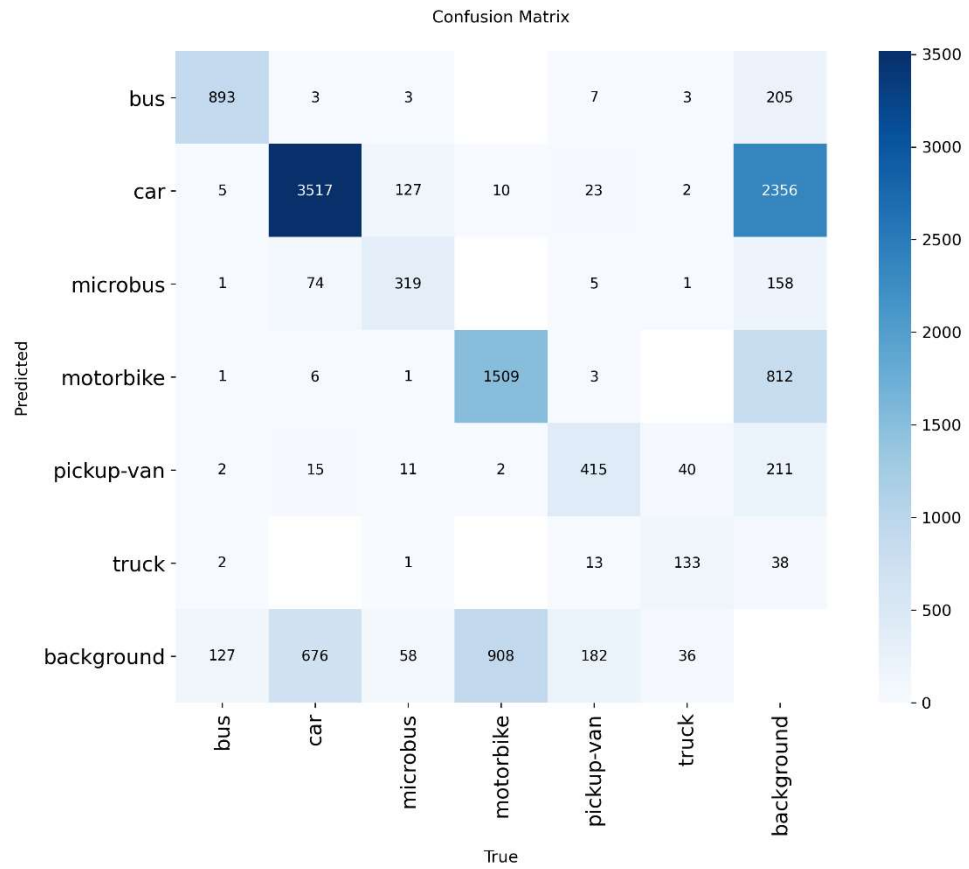


Figure 8. Confusion Matrix - YOLOv8n

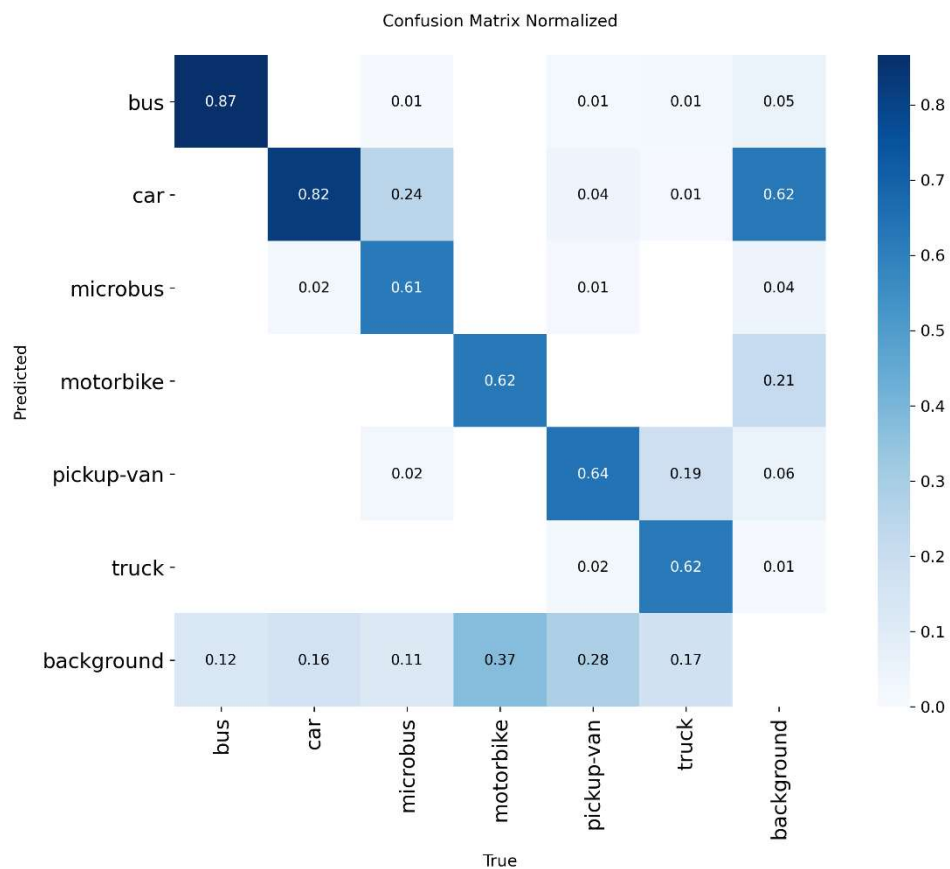


Figure 9. Confusion Matrix Normalized - YOLOv8n

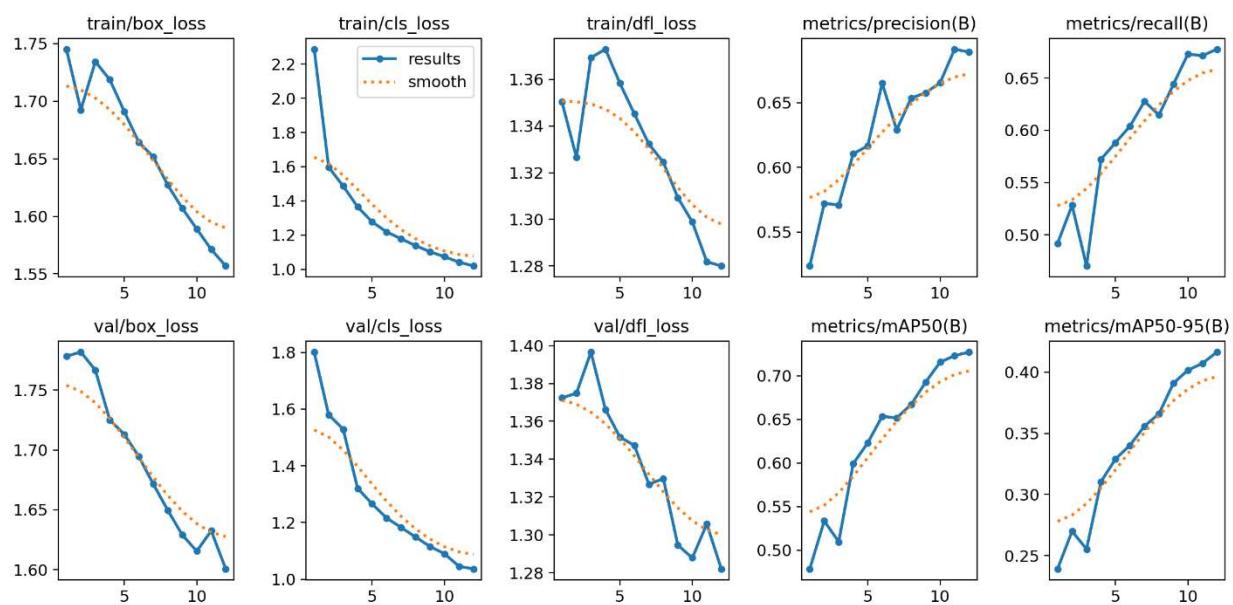


Figure 10. YOLOv8n Metrics

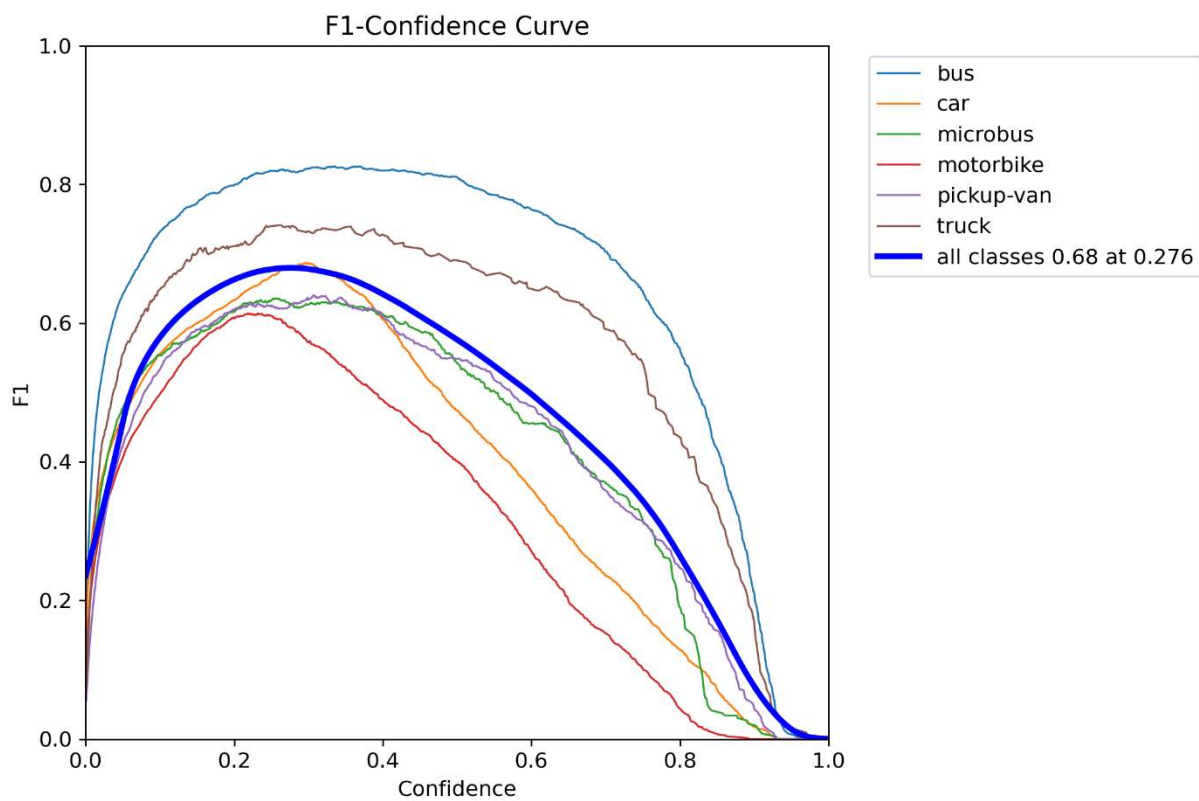


Figure 11. YOLOv8n F1 Curve

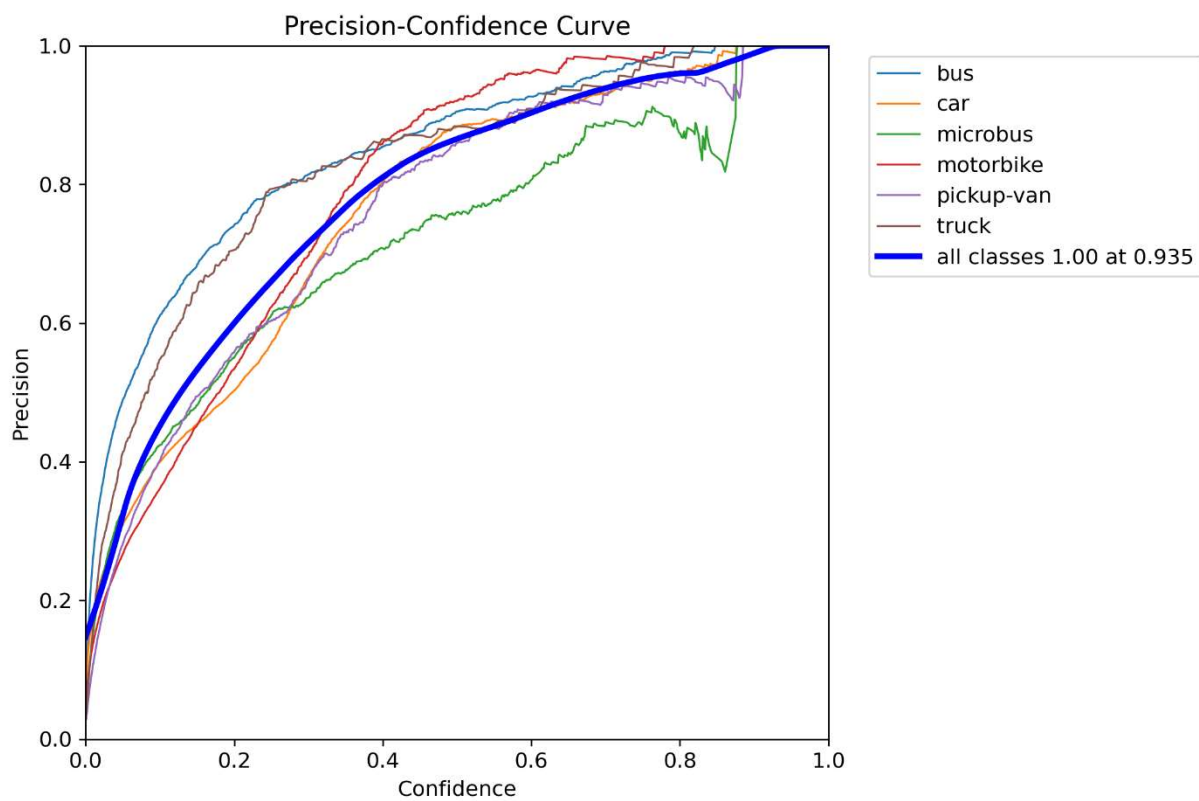


Figure 12. YOLOv8n Precision Curve

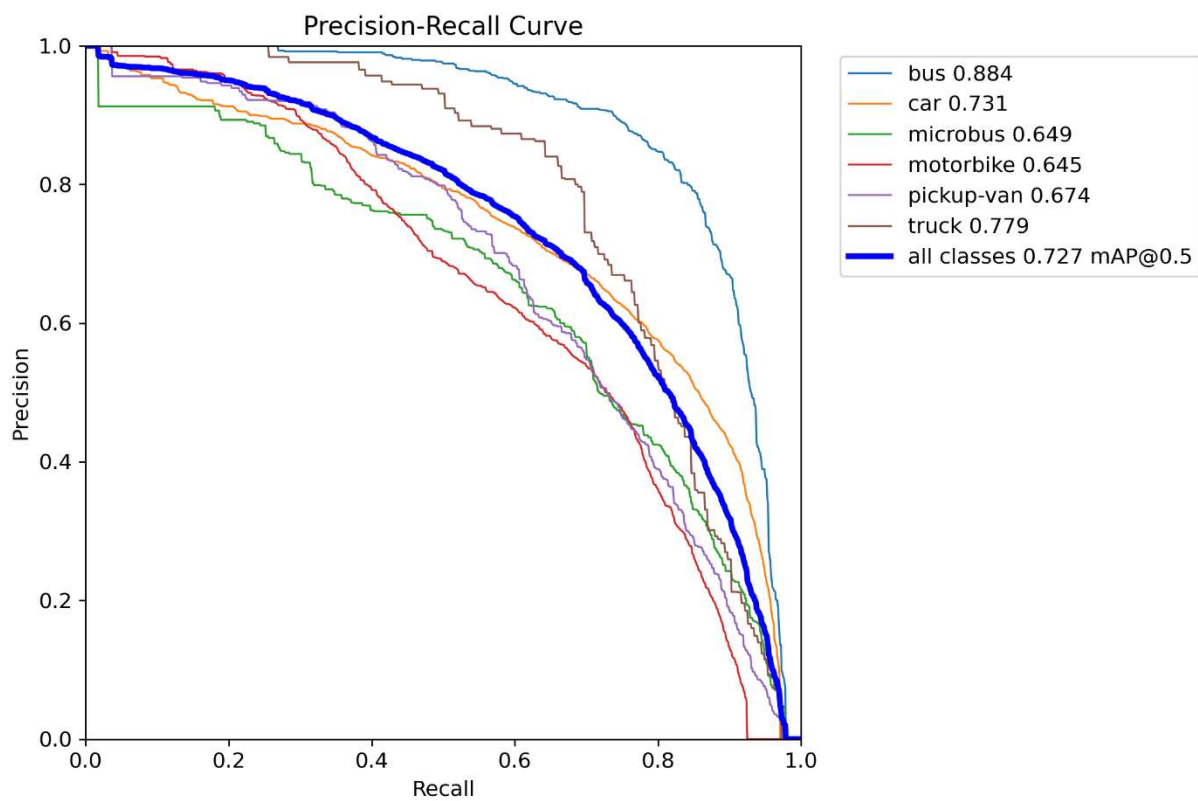


Figure 13. YOLOv8n Precision-Recall Curve

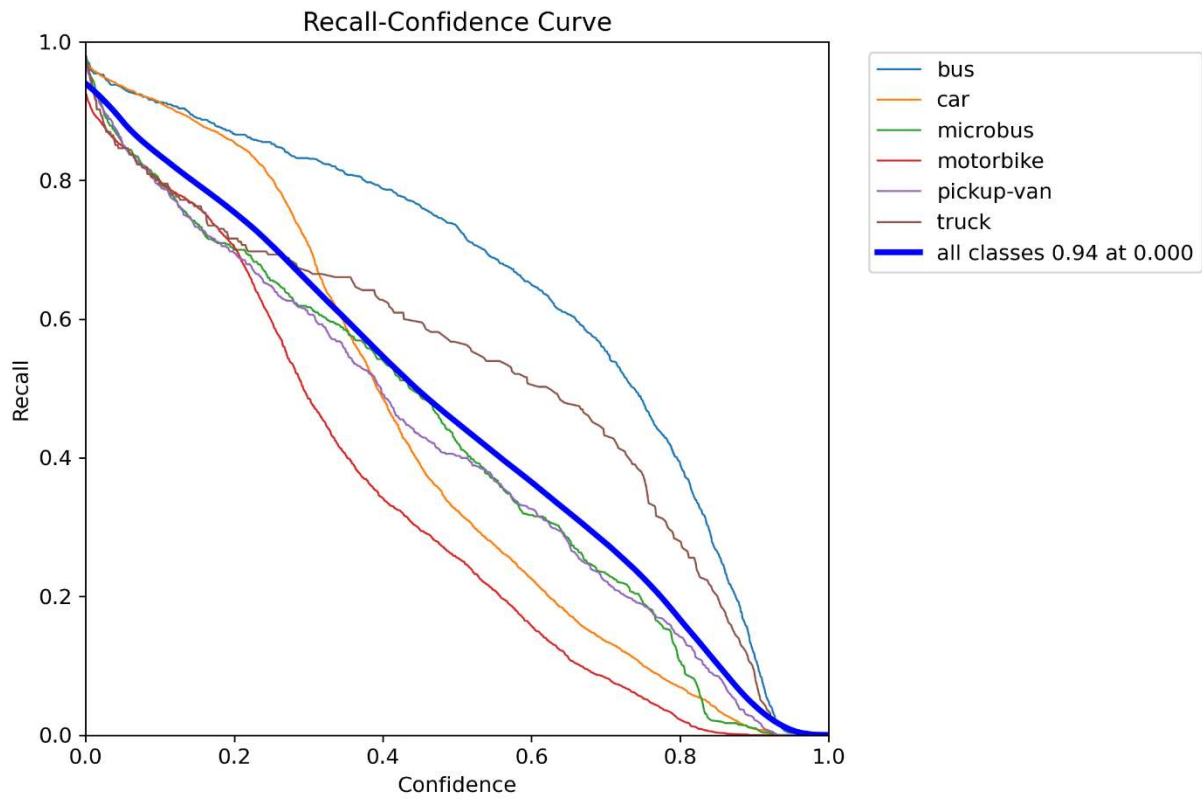


Figure 14. YOLOv8n Recall Curve

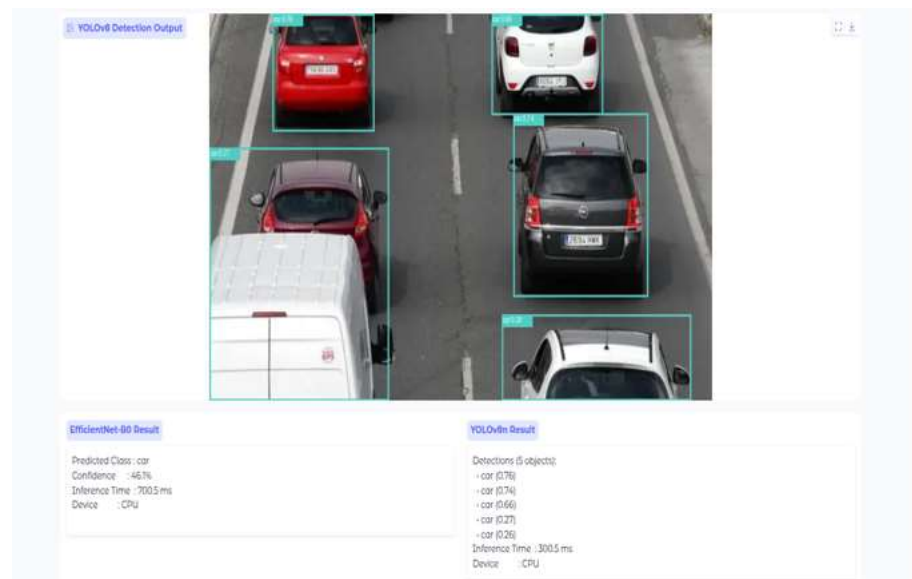


Figure 15. Gradio Inference - Phase 5

Additional Plots



Figure 16. YOLOv8n Predictions on test batch 1



Figure 17. YOLOv8n Predictions on test batch 2

References

- [1]. <https://universe.roboflow.com/lynkeus03/vehicle-detection-by9xs>
- [2]. EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks, M. Tan and Q. Le, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," in *Proceedings of the 36th International Conference on Machine Learning (ICML)*, Long Beach, CA, USA, 2019, pp. 6105–6114.
- [3]. You Only Look Once: Unified, Real-Time Object Detection, Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, USA, 2016, pp. 779–788.
- [4]. Ultralytics YOLO, *Ultralytics YOLO Documentation*. Ultralytics, 2023–present. [Online]. Available: <https://docs.ultralytics.com>. [Accessed: Jun. 30, 2026].